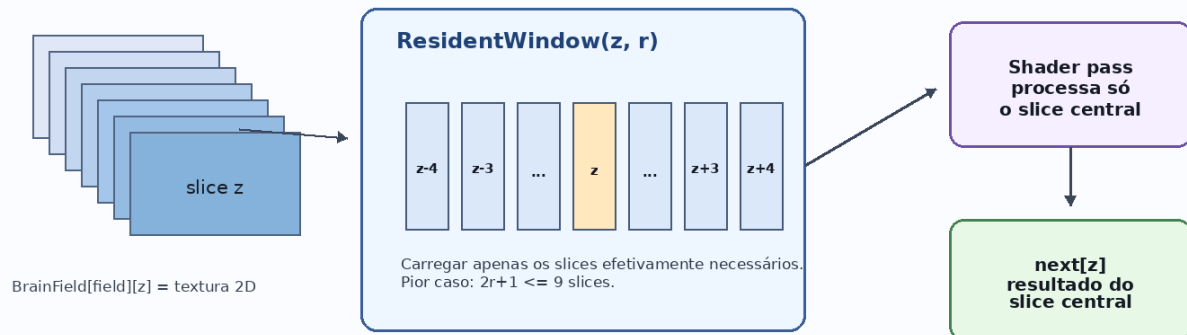


Migracao do cerebro 3D para shaders

Especificacao curta para preservar a modelagem organica do projeto em uma pilha indexada de texturas por slice, com janela residente sob demanda e ping-pong.

Arquitetura: pilha indexada de slices + janela residente



Nota: Texture2DArray global nao e a representacao principal. O volume completo fica paginado em slices/tiles.

Resumo. O cerebro logico permanece um volume 3D. O backend passa a armazenar cada campo neural como uma stack indexada de slices/tiles 2D. Para atualizar o slice z, o shader recebe apenas os slices necessarios dentro do raio vertical maximo, le pontos via `texture(other_slice_texture, uv)` ou `texelFetch`, e escreve somente o proximo estado do slice central. O objetivo nao e transformar o modelo em um automato fixo; e preservar regioes locais, conectividade dirigida, plasticidade e heranca espacial.

Base conceitual do projeto atual: genoma -> cerebro 3D -> inputs/outputs -> thresholds, regioes, conexoes, forcas dinamicas, reproducao e mutacao. A migracao abaixo troca dicionarios/listas variaveis em CPU por texturas alinhadas por neuronio/pixel.

1. Representacao principal

A representacao principal deve ser uma stack indexada de texturas por campo neural, nao uma Texture2DArray global com o cerebro inteiro. Texture2DArray pode existir apenas como otimizacao local para uma janela pequena; a unidade de paginacao continua sendo o slice ou tile.

```
BrainField[field][z] = textura 2D do slice z
BrainField[field][z][tile_x][tile_y] = textura 2D se o slice for grande demais
```

Para processar z:
 bind apenas z + dz para dz realmente usado
 dz fica limitado por MAX_Z_RADIUS, por exemplo 4
 pior caso: $2 * \text{MAX_Z_RADIUS} + 1 = 9$ slices

Regras de residencia

- A chamada `texture(other_slice_texture, uv)` le apenas a coordenada solicitada, mas a textura do slice precisa estar carregada/bindada como recurso GPU.
- O sistema de Godot/GDScript, nao o shader, decide quais slices entram no conjunto residente antes do pass.
- Se um slice inteiro estourar a VRAM, a solucao e subdividir em tiles 2D mantendo a mesma logica: slice-indexed + tile-indexed.

Um slice anatomico vira um conjunto alinhado de texturas



Campos fixos/geneticos nao duplicam. Campos dinamicos usam ping-pong quando escritos.

2. O que vira textura

Nem tudo deve virar textura. Metadados globais, selecao evolutiva e reproducao podem continuar no lado CPU/engine. O que deve virar textura e o estado por neuronio/pixel e a estrutura que precisa ser lida no shader.

Elemento do modelo atual	Textura/campo proposto	Ping-pong ?	Observacao
neuron_signals	signal_cur[z], signal_next[z]	Sim	Ativacao/sinal acumulado do neuronio. R16F ou R32F.
disparo acima do threshold	emit_cur[z]	Nao; derivado	Textura de emissao. Ler emit dos slices vizinhos em vez de signal bruto.
thr, decay, exp_response	params[z]	Nao	RGBA16F: threshold, decay, exp_k, excitability/tipo.
inactive_counter, weakened, traces	state_cur[z], state_next[z]	Sim	RGBA16F: refratario, weakened, trace curto, modulacao/fadiga.
regions + probs	region_growth[z]	Nao	RGBA8/16: radius_x, radius_y, radius_z, growth_prob.
connections	conn_x_g, conn_y_g, conn_zmeta_g	Nao em v1	Slots candidatos fixos. Ativo se weight > eps.
connection_strengths	weight_g_cur[z], weight_g_next[z]	Sim	Forca sinaptica por slot; decai/reforca durante a vida.
estado de plasticidade	plasticity_g_cur[z], plasticity_g_next[z]	Sim	Eligibility/stability/age por slot, se usado.
inputs	io_map ou lista de coordenadas	Depende	Pode permanecer como lista; injeta sinais escrevendo em signal_cur ou input_tex.
outputs + outputs_thr	io_map/output list	Depende	Pode permanecer lista; le coordenadas de saida apos os passes.
dims	metadata do BrainVolume	Nao	Define largura, altura, numero de slices e tileamento.

Escolha importante: guardar conexoes como slots de entrada do neuronio alvo. Assim, ao processar o slice z, o shader le pesos/offsets apenas do slice central e le dos slices vizinhos somente a textura emit_cur. Isso evita scatter write e reduz a janela residente.

3. Codificacao das conexoes

Cada neuronio/pixel possui K slots candidatos de entrada. Um slot representa uma conexao dirigida de uma fonte local para o neuronio alvo atual. A fonte e calculada por offset relativo:

```
target = (x, y, z)
slot k:
    source = (x + dx[k], y + dy[k], z + dz[k])
    weight = weight[k]
    active = weight > EPSILON and meta.enabled
```

Empacotamento recomendado por grupos de 4 slots

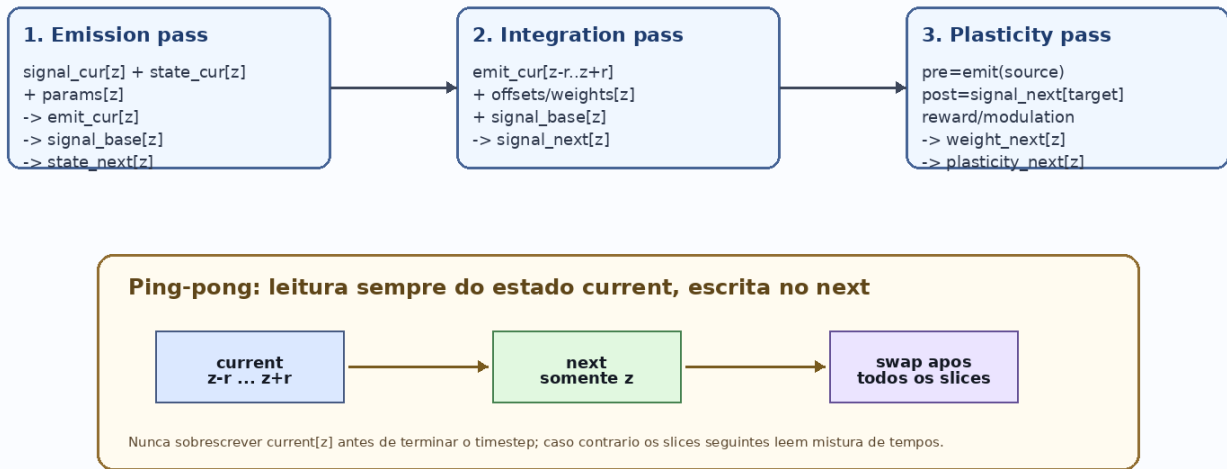
Textura por grupo g	Formato inicial	Canal R/G/B/A	Decodificacao
conn_x_g[z]	RGBA8 ou RGBA16	dx dos slots 4g..4g+3	dx = code - MAX_DX
conn_y_g[z]	RGBA8 ou RGBA16	dy dos slots 4g..4g+3	dy = code - MAX_DY
conn_zmeta_g[z]	RGBA8	dz + enabled + tipo/sinal	0=disabled; senao dz=(code-1)%NZ - RZ
weight_g_cur/next[z]	RGBA16F	peso dos 4 slots	ativo se peso > EPSILON
plasticity_g_cur/next[z]	RGBA16F	eligibility ou estabilidade	opcional; adicionar plast2_g se precisar mais estado

Por que slots candidatos preservam o modelo original

- O genoma ainda define regioes locais, possiveis conexoes, probabilidades e parametros por neuronio.
- Nascimento de conexao vira ativacao de um slot dormente: o offset ja existe, mas o peso sobe de zero quando a plasticidade favorece.
- Remocao de conexao vira decaimento do peso para zero. A estrutura genetica continua estavel durante a vida, mas a rede efetiva muda.
- Se for indispensavel mudar o proprio offset durante a vida, criar conn_*_next. Eu nao recomendo na primeira versao porque dobra texturas estruturais e dificulta sincronismo.

4. Como o shader deve processar

Ciclo por timestep: emission, integration/plasticity e swap



A implementacao mais fiel usa dois ou tres passes por timestep. Isso evita que um slice atualizado contamine outro no mesmo tempo logico.

Pass	Le	Escreve	Finalidade
Emission	signal_cur[z], state_cur[z], params[z]	emit_cur[z], signal_base[z], state_next[z]	Decide quem disparou, aplica refratario, zera/decai sinal proprio.
Integration	emit_cur[z+dz], conn_*/weight[z], signal_base[z]	signal_next[z]	Soma entradas de slots candidatos e gera novo sinal.
Plasticity	emit_cur[z+dz], signal_next[z], weight_cur[z], plasticity_cur[z]	weight_next[z], plasticity_next[z]	Refinca, enfraquece, ativa ou desativa conexoes efetivas.

Uso de Godot

- No projeto atual o renderer esta em GL Compatibility. Compute shaders em Godot exigem renderers baseados em RenderingDevice, isto e Forward+ ou Mobile; portanto, com a configuracao atual, comece com CanvasItem shaders renderizando em Viewports offscreen.
- Se migrar para Forward+/Mobile, compute shaders passam a ser a rota mais limpa para multiplos outputs e storage buffers/textures.
- Para dados neuronais discretos, usar filtering nearest, mipmaps off, repeat disabled/clamp. Se disponivel no pipeline, preferir texelFetch; caso contrario, usar texture() com nearest e UV centralizado no pixel.

5. Exemplo minimo de shader Godot

Exemplo conceitual de integration pass para um slice central. O codigo mostra a ideia; na versao real, repetir para todos os grupos g e todos os slots dentro do grupo. Uniforms nao usados podem apontar para uma textura dummy preta.

```
shader_type canvas_item;

uniform sampler2D emit_m4;
uniform sampler2D emit_m3;
uniform sampler2D emit_m2;
uniform sampler2D emit_m1;
uniform sampler2D emit_0;
uniform sampler2D emit_p1;
uniform sampler2D emit_p2;
uniform sampler2D emit_p3;
uniform sampler2D emit_p4;

uniform sampler2D signal_base;          // slice z
uniform sampler2D params_tex;           // slice z
uniform sampler2D conn_x0;              // 4 slots: R,G,B,A
uniform sampler2D conn_y0;
uniform sampler2D conn_zmeta0;
uniform sampler2D weight0_cur;

uniform vec2 texel_size;
uniform int MAX_DX = 127;
uniform int MAX_DY = 127;
uniform int RZ = 4;
uniform float EPSILON = 0.0001;

int u8(float v) { return int(round(clamp(v, 0.0, 1.0) * 255.0)); }
int decode_dx(float v) { return u8(v) - MAX_DX; }
int decode_dy(float v) { return u8(v) - MAX_DY; }

float read_emit(int dz, vec2 uv) {
    if (dz == -4) return texture(emit_m4, uv).r;
    if (dz == -3) return texture(emit_m3, uv).r;
    if (dz == -2) return texture(emit_m2, uv).r;
    if (dz == -1) return texture(emit_m1, uv).r;
    if (dz == 0) return texture(emit_0, uv).r;
    if (dz == 1) return texture(emit_p1, uv).r;
    if (dz == 2) return texture(emit_p2, uv).r;
    if (dz == 3) return texture(emit_p3, uv).r;
    return texture(emit_p4, uv).r;
}

float slot_sum(float dxv, float dyv, float metav, float w, vec2 uv) {
    int meta = u8(metav);
    if (meta == 0 || w <= EPSILON) return 0.0;

    int nz = 2 * RZ + 1;
    int dz = ((meta - 1) % nz) - RZ;
    int dx = decode_dx(dxv);
    int dy = decode_dy(dyv);

    vec2 src_uv = uv + vec2(float(dx), float(dy)) * texel_size;
    return read_emit(dz, src_uv) * w;
}

void fragment() {
    vec4 dx4 = texture(conn_x0, UV);
    vec4 dy4 = texture(conn_y0, UV);
    vec4 mt4 = texture(conn_zmeta0, UV);
    vec4 w4 = texture(weight0_cur, UV);

    float incoming = 0.0;
    incoming += slot_sum(dx4.r, dy4.r, mt4.r, w4.r, UV);
    incoming += slot_sum(dx4.g, dy4.g, mt4.g, w4.g, UV);
    incoming += slot_sum(dx4.b, dy4.b, mt4.b, w4.b, UV);
    incoming += slot_sum(dx4.a, dy4.a, mt4.a, w4.a, UV);

    vec4 p = texture(params_tex, UV);
    float threshold = p.r;
    float base = texture(signal_base, UV).r;

    float next_signal = max(0.0, base + incoming - threshold);
    COLOR = vec4(clamp(next_signal, 0.0, 1.0), 0.0, 0.0, 1.0);
}
```

Observacao: se usar texelFetch, trocar UV por coordenadas inteiras. Para CanvasItem/Viewports, texture() com nearest e sem mipmaps e suficiente para prototipar.

6. Ping-pong e custo de memoria

Orcamento de VRAM: limite pela janela residente

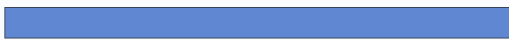
```

M_residente ~= n_z * M_emit_cur
              + 1 * M_signal_next
              + 2 * (M_weight_central + M_plasticity_central + M_state_central)
              + 1 * (M_params + M_region + M_offsets + M_io)

n_z <= 2*r + 1 <= 9

```

A formula assume conexoes de entrada armazenadas no slice-alvo; fontes vizinhas fornecem apenas emit_cur.

emit_cur window		
signal_next		1x
dynamic central x2		2x
fixed central		1x

Se o slice inteiro nao couber,
dividir em tiles:
BrainField[z][tile_x][tile_y]

O ping-pong dobra somente os campos que sao escritos sincronicamente. Ele nao dobra os campos fixos/geneticos e nao exige manter next para toda a janela se voce escreve apenas o slice central.

Para atualizar slice z:

```

leitura cross-slice:
    emit_cur[z-r ... z+r]      -> ate 9 slices
leitura/escrita central:
    signal_base[z] -> signal_next[z]
    state_cur[z]   -> state_next[z]
    weight_cur[z]  -> weight_next[z]
    plast_cur[z]   -> plast_next[z]
fixos centrais:
    params[z], region[z], conn_*[z], io_map[z]

```

Nao fazer:

```
current[z] = next[z] imediatamente
```

Fazer:

```

processar todos os z gerando next[z]
depois swap current <-> next

```

Formula pratica

Se $S(\text{field})$ e o custo de um campo para um slice/tile, e n_z e o numero de slices residentes usados naquele pass, entao:

```

M_residente ~= n_z * S(emit_cur)
              + S(signal_base) + S(signal_next)
              + S(state_cur)  + S(state_next)
              + G * (S(weight_cur) + S(weight_next))
              + G * (S(plast_cur)  + S(plast_next))
              + S(params + region + io)
              + G * S(conn_x + conn_y + conn_zmeta)

```

$G = \text{ceil}(K / 4)$, onde K = slots candidatos por neuronio.
 $n_z \leq 9$ no desenho discutido, mas pode ser 1, 3, 5, 7 ou 9.

Use essa formula antes de escolher o tamanho do slice/tile. Se o resultado ultrapassar a VRAM alvo, reduza o tile em x/y ou reduza K/formato dos campos dinamicos. Nao altere a teoria do cerebro por causa disso; altere a granularidade de paginação.

7. Reproducao, heranca e ciclo de engine

Heranca

- Texturas geneticas/herdaveis: params, region_growth, conn_x/y/zmeta, base_weights se voce quiser iniciar pesos herdados.
- Texturas de estado aprendido: signal, emit, state, plasticity traces e pesos modificados durante a vida. Por padrao Baldwiniano, nao herdar estas memorias; reiniciar neutro no filho.
- Crossover espacial: copiar retangulos/tiles/slices coerentes dos pais para o filho, mantendo alinhamento entre todos os campos herdaveis.
- Mutacao: operar nos texels herdaveis, preservando restricoes de raio, dz maximo, tipo e limites de formato.

Ciclo por frame neural

```

for each brain:
  for each z/tile:
    ensure resident fields for emission of z
    run emission pass -> emit_cur[z], signal_base[z], state_next[z]

  for each z/tile:
    discover/bind needed emit_cur[z+dz]
    bind central params/conn/weights/plasticity
    run integration pass -> signal_next[z]
    run plasticity pass -> weight_next[z], plasticity_next[z]

  after all slices/tiles:
    swap signal_cur/signal_next
    swap state_cur/state_next
    swap weight_cur/weight_next
    swap plasticity_cur/plasticity_next
    discard/reuse emit_cur and signal_base temporaries

```

Checklist de implementacao

- Implementar primeiro um cerebro pequeno com 3 slices e K=4 slots para validar equivalencia temporal.
- Desligar mipmaps e filtro linear em todos os campos neurais; usar nearest/clamp.
- Validar ping-pong comparando uma versao CPU pequena contra a versao shader por algumas iteracoes.
- Medir VRAM por campo antes de aumentar slice/tile; registrar n_z real usado por pass.
- Manter log visual: renderizar signal_cur, emit_cur, weight_cur e plasticity para slices escolhidos.

8. Decisoes finais da migracao

Tema	Decisao
Representacao global	Stack indexada de texturas por slice/tile. Nao usar Texture2DArray global gigante.
Unidade de processamento	Um slice/tile central por pass. Le janela local conforme dz realmente usado.
Conectividade	Slots candidatos de entrada por neuronio alvo, com offsets dx/dy/dz dentro de raio genetico.
Plasticidade estrutural	Ativar/desativar conexoes efetivas por peso > epsilon. Offsets permanecem como candidatos geneticos na v1.
Cross-slice sampling	texture(other_slice_texture, uv) ou texelFetch em slices residentes. Nunca implica streaming automatico.
Sincronismo	Obrigatorio ping-pong current/next para campos dinamicos escritos.
Escala extrema	Se slice inteiro nao cabe, quebrar em tiles. Pior caso local: ate 9 slices anatomicos vezes campos necessarios.
Godot	Com GL Compatibility atual, usar CanvasItem shaders + Viewports. Compute exige Forward+/Mobile.

Conclusao. A migracao preserva a tese original se o shader for tratado como um mecanismo de atualizacao local de um tecido neural 3D, nao como um stencil fixo global. O desenho recomendado e: campos por slice, slots locais dirigidos, leitura pontual de slices residentes, ping-pong sincrono e custo limitado por janela/tile.

Referencias de implementacao

- Godot Texture2DArray: https://docs.godotengine.org/en/stable/classes/class_texture2darray.html
- Godot CanvasItem shaders:
https://docs.godotengine.org/en/stable/tutorials/shaders/shader_reference/canvas_item_shader.html
- Godot Compute shaders: https://docs.godotengine.org/en/latest/tutorials/shaders/compute_shaders.html
- Projeto Neurons anexado nesta conversa: geracao genetica de cerebro 3D, inputs/outputs, regioes, conexoes, thresholds, decay e reproducao.